# Programación Orientada a Objetos II - Abstracción y modularización

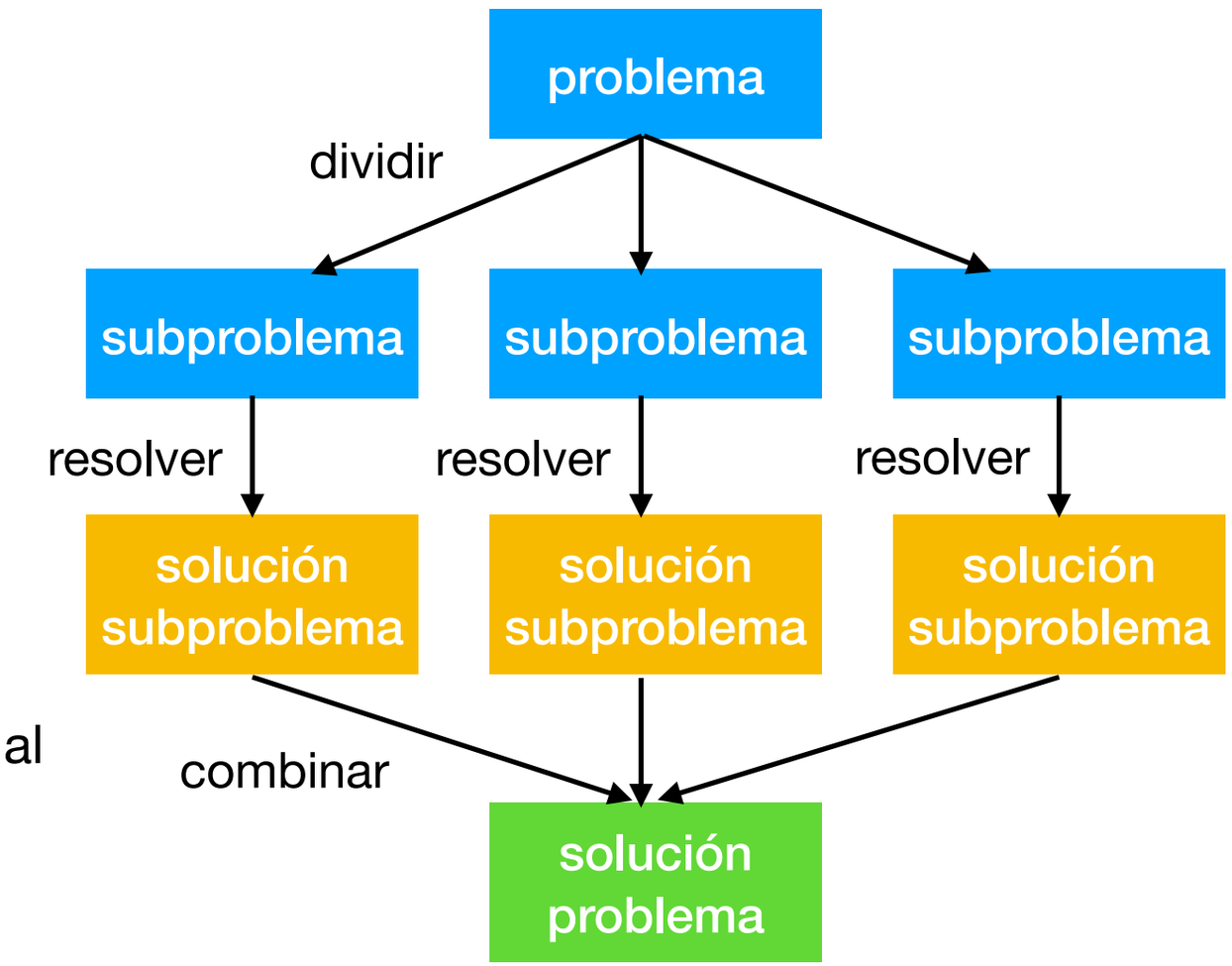
Algoritmos y Estructuras de Datos II -
Estructuras de Datos y Algoritmos

Dr. Pablo Ponzio
Universidad Nacional de Río Cuarto
CONICET

CREER... CREAR... CRECER

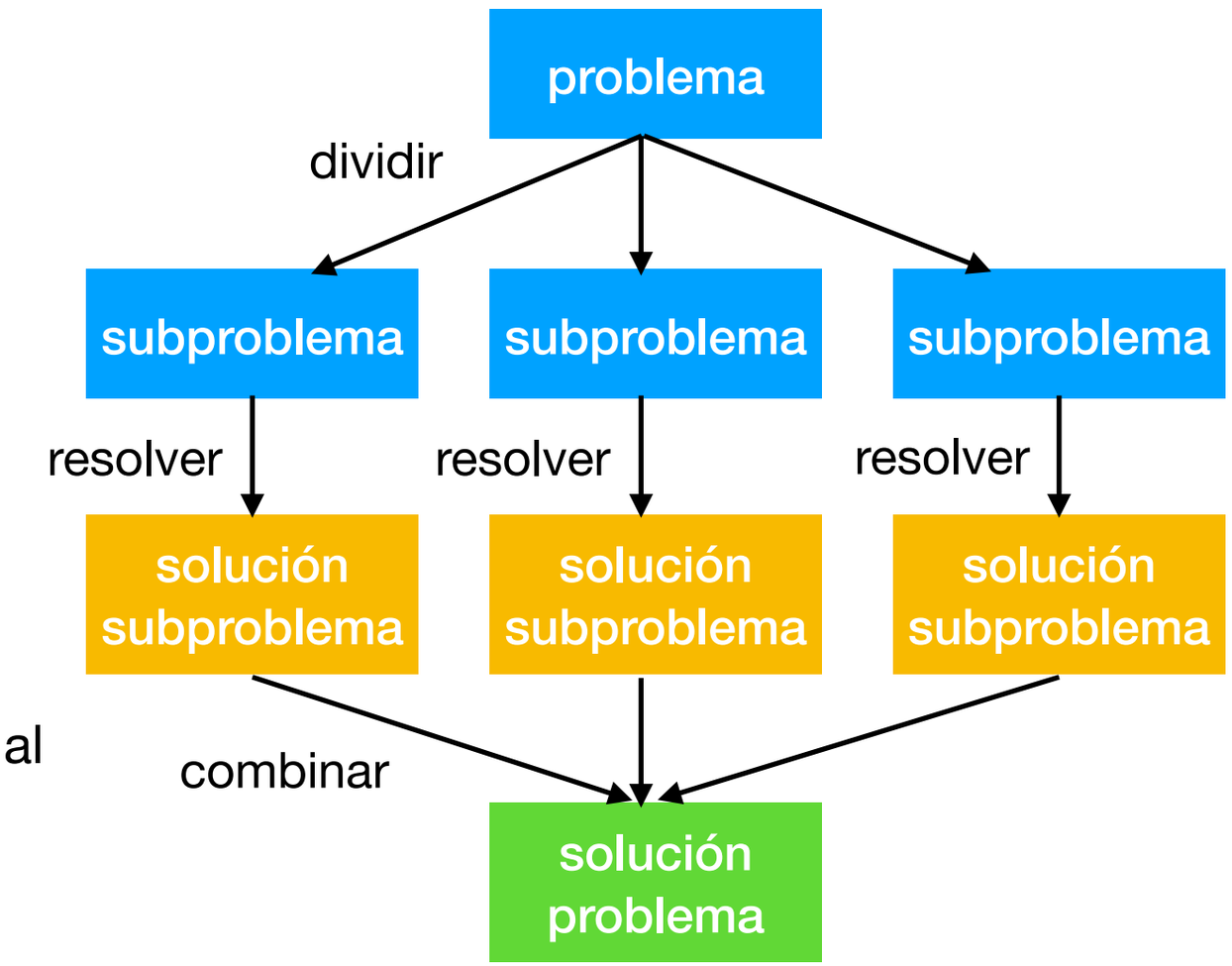
# Divide and conquer

- Cuando los problemas son muy complejos, es muy difícil tener en cuenta todos los detalles relevantes del problema todo el tiempo
- Para lidiar con la complejidad usamos la técnica conocida como divide-and-conquer (divide y reinarás)
- Dividimos el problema en subproblemas más fáciles de resolver que el problema original
- Resolvemos cada subproblema por separado, concentrándonos sólo en los detalles relevantes al subproblema en cuestión
- Finalmente, combinamos las soluciones de los subproblemas para dar solución al problema original
    - En la combinación, tratamos la solución de cada subproblema como una unidad; nos abstraemos de sus detalles



# Divide and conquer

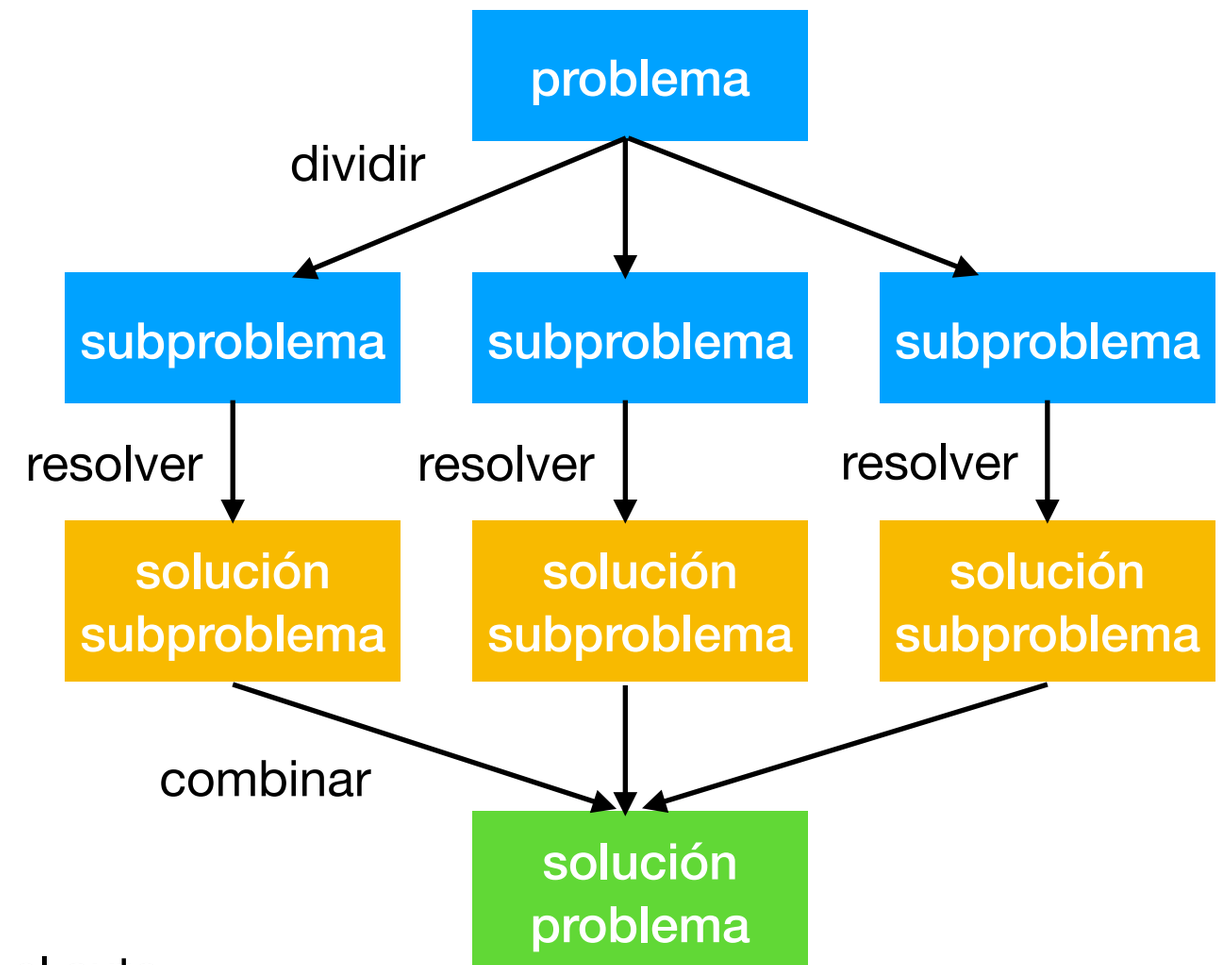
- Cuando los problemas son muy complejos, es muy difícil tener en cuenta todos los detalles relevantes del problema todo el tiempo
- Para lidiar con la complejidad usamos la técnica conocida como divide-and-conquer (divide y reinarás)
- Dividimos el problema en subproblemas más fáciles de resolver que el problema original
- Resolvemos cada subproblema por separado, concentrándonos sólo en los detalles relevantes al subproblema en cuestión
- Finalmente, combinamos las soluciones de los subproblemas para dar solución al problema original




- Esta técnica puede aplicarse repetidamente: cada subproblema a su vez puede dividirse en subproblemas más pequeños que se resuelven por separado, y así sucesivamente

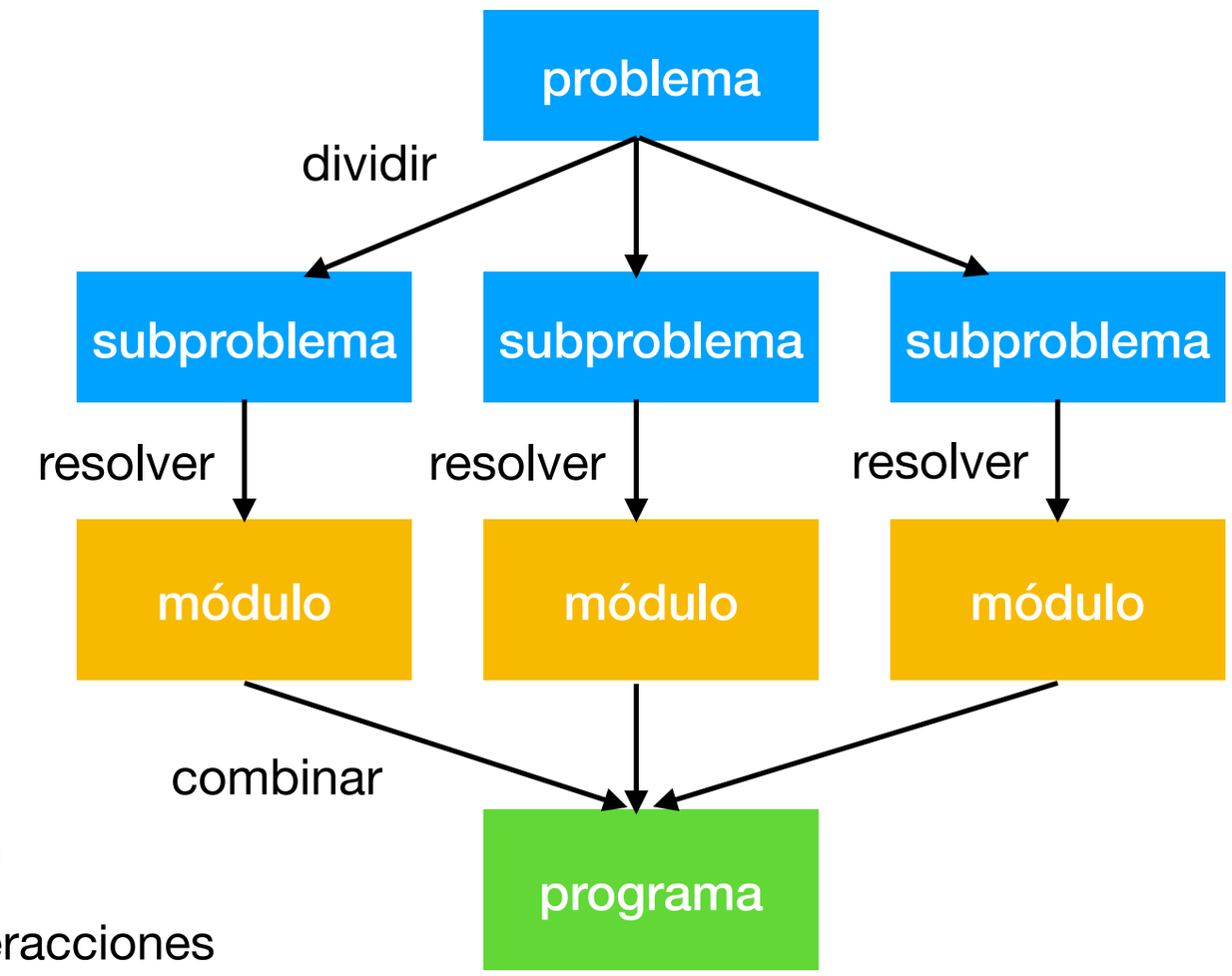
# Abstracción y Modularización

- Ejemplo: Para fabricar un auto es imposible para una sola persona conocer cada detalle de cada parte del auto
- Los autos se fabrican exitosamente porque los ingenieros usan la estrategia de divide-and-conquer, y los principios de modularización y abstracción
- El auto se divide en módulos independientes entre sí: la carrocería, el motor, las ruedas, los asientos, etc.
    - Distintas empresas trabajan independientemente en los distintos módulos



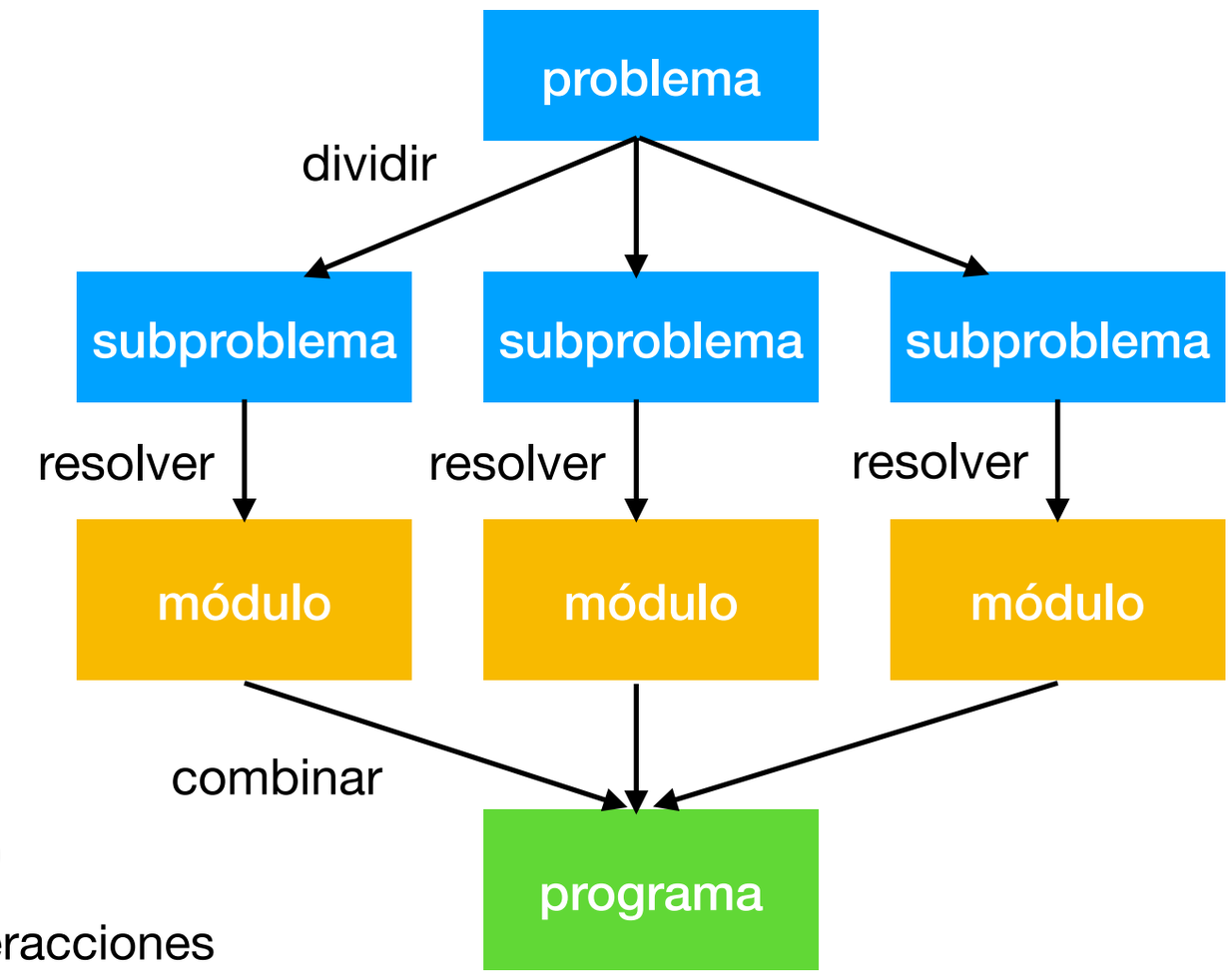

- Finalmente, los módulos se combinan para construir el auto
    - En este punto, se usa abstracción: se ve a cada módulo como una unidad, ignorando los detalles de su fabricación
    - Ej: Cuando ensamblamos el auto nos abstraemos de los detalles de los materiales que se usaron para hacer los neumáticos, el proceso usado para su fabricación, etc.

# Abstracción y Modularización




- Modularización: Proceso de dividir un todo en partes bien definidas, que pueden ser construidas y examinadas por separado, y que interactúan de manera bien definida
- Abstracción: es la capacidad de ignorar los detalles de las partes, para centrar la atención en un nivel superior de un problema
- Los principios de modularización y abstracción son cruciales en el desarrollo de software
- Para desarrollar un programa complejo identificamos subcomponentes que podemos desarrollar en módulos independientes (en lo posible)
- El programa completo estará conformado por las interacciones entre los distintos módulos
    - Usamos los servicios que proveen los módulos, ignorando su complejidad interna (abstracciones de datos)
- En programación orientada a objetos, los módulos están conformados por conjuntos de objetos

# Abstracción y Modularización




- Modularización: Proceso de dividir un todo en partes bien definidas, que pueden ser construidas y examinadas por separado, y que interactúan de manera bien definida
- Abstracción: es la capacidad de ignorar los detalles de las partes, para centrar la atención en un nivel superior de un problema
- Los principios de modularización y abstracción son cruciales en el desarrollo de software
- Para desarrollar un programa complejo identificamos subcomponentes que podemos desarrollar en módulos independientes (en lo posible)
- El programa completo estará conformado por las interacciones entre los distintos módulos
  - Usamos los servicios que proveen los módulos, ignorando su complejidad interna (abstracciones de datos)
- En

**Identificar las clases y objetos que componen (los módulos de) un sistema de software es una tarea crucial para desarrollar software de calidad**

# Ejemplo: ClockDisplay

- Para nuestro primer ejemplo de modularización, vamos a implementar un reloj de 24 horas
  - El display muestra números entre 00:00 y 23:59
- Una forma de implementar el reloj sería representar los cuatro dígitos por separado
- Sin embargo, observemos que los dígitos de las horas y los de los minutos se comportan de manera similar
  - Las horas comienzan en 0, se van incrementando en 1, y cuando llegan a su límite (24) se resetean a 0
  - Los minutos comienzan en 0, se van incrementando en 1, y cuando llegan a su límite (60) se resetean a 0
- Este comportamiento similar nos indica que podemos aplicar abstracción
  - Considerar a los minutos y a las horas como instancias de un mismo módulo: Un display de dos dígitos que va desde 0 a un límite dado
  - Podemos aprovechar esta abstracción también para agregar los segundos al reloj

# Abstracción de datos: `NumberDisplay`

```java
/**
 * The NumberDisplay class represents a digital number display that can hold
 * values from zero to a given limit. The limit can be specified when creating
 * the display. The values range from zero (inclusive) to limit-1. When
 * incremented, the display automatically rolls over to zero when reaching the limit.
 */
public class NumberDisplay
{
    /**
     * @pre {@literal 'rollOverLimit' > 0}
     * @post Constructs a NumberDisplay that starts at 0
     *     and resets at the given 'rollOverLimit'.
     */
    public NumberDisplay(int rollOverLimit)
    /**
     * @post Return the display value, that is, the current value
     *     as a two-digit String.
     */
    public String getDisplayValue()
    /**
     * @post Return the current value.
     */
    public int getValue()
    /**
     * @post Increment the display value by one, rolling over to zero if the
     * limit is reached.
     */
    public void increment()
}
```

# Abstracción de datos: `NumberDisplay`

```
/**
 * The NumberDisplay class represents a digital number display that can hold
 * values from zero to a given limit. The limit can be specified when creating
 * the display. The values range from zero (inclusive) to limit-1. When
 * incremented, the display automatically rolls over to zero when reaching the limit.
 */
public class NumberDisplay
{
    /**
     * @pre {@literal 'rollOverLimit' > 0}
     * @post Constructs a NumberDisplay that starts at 0
     *     and resets at the given 'rollOverLimit'.
     */
    public NumberDisplay(int rollOverLimit)
    /**
     * @post Return the display value, that is, the current value
     *     as a two-digit String.
     */
    public String getDisplayValue()
    /**
     * @post Return the current value.
     */
    public int getValue()
    /**
    public void increment()
}
```

Aquí tenemos la especificación y las funcionalidades más importantes de `NumberDisplay`

# Implementación de `NumberDisplay`

```java
public class NumberDisplay
{
    private int limit;
    private int value;
    /**
     * @pre {@literal 'rollOverLimit' > 0}
     * @post Constructs a NumberDisplay that starts at 0
     * and resets at the given 'rollOverLimit'.
     */
    public NumberDisplay(int rollOverLimit)
    {
        limit = rollOverLimit;
        value = 0;
    }
    /**
     * @post Return the current value.
     */
    public int getValue()
    {
        return value;
    }
   /** ... Continúa ... */
```

# Implementación de `NumberDisplay`

```java
public class NumberDisplay
{
    private int limit;
    private int value;
    /**
     * @pre {@literal 'rollOverLimit'
     * @post Constructs a NumberDispla
     * and resets at the given 'rollOv
     */
    public NumberDisplay(int rollOverI
    {
        limit = rollOverLimit;
        value = 0;
    }
    /**
     * @post Return the current value.
     */
    public int getValue()
    {
        return value;
    }
   /** ... Continúa ... */
```

```java
    /**
     * @post Return the display value, that is,
     * the current value as a two-digit String.
     */
    public String getDisplayValue()
    {
        if(value < 10) {
            return "0" + value;
        }
        else {
            return "" + value;
        }
    }
    /**
     * @post Increment the display value by one,
     * rolling over to zero if the limit is reached.
     */
    public void increment()
    {
        value = (value + 1) % limit;
    }
}
```

# Ejemplo: ClockDisplay

- Hasta aquí definimos nuestro módulo `NumberDisplay`, encargado de modelar displays de dos dígitos
- Este módulo consiste de una única clase `NumberDisplay`
- Recordar que:
  - Las clases definen tipos
  - Los elementos del tipo son objetos: las instancias de la clase
  - Podemos definir variables para almacenar referencias a objetos del tipo

```
/**
 * The NumberDisplay class represents a digita
 * values from zero to a given limit. The limi
 * the display. The values range from zero (in
 * incremented, the display automatically roll
 */
public class NumberDisplay
{
    /**
     * @pre {@literal 'rollOverLimit' > 0}
     * @post Constructs a NumberDisplay that s
     *     and resets at the given 'rollOverLi
     */
    public NumberDisplay(int rollOverLimit)
    /**
     * @post Return the display value, that is
     *     as a two-digit String.
     */
    public String getDisplayValue()
    /**
     * @post Return the current value.
     */
    public int getValue()
    /**
     * @post Increment the display value by on
     * limit is reached.
     */
    public void increment()
}
```

# Ejemplo: ClockDisplay

- Ahora estamos listos para definir `ClockDisplay`
- `ClockDisplay` usa las funcionalidades provistas por el módulo `NumberDisplay`:
  - `ClockDisplay` tiene dos atributos de tipo `NumberDisplay`, uno para representar las horas y otro para los minutos
- En Java, los atributos son de tipo objeto guardan una referencia a un objeto
  - A diferencia de los atributos de tipo primitivo, que simplemente almacenan valores del tipo
  - En el ejemplo, `hours` y `minutes` hacen referencia a objetos de tipo `NumberDisplay`
- Inicialmente las variables que almacenan objetos toman el valor especial `null`
  - Esto indica que no hay memoria reservada para un objeto

```java
public class ClockDisplay
{
    private NumberDisplay hours;
    private NumberDisplay minutes;
    // Código omitido
}
```

# Abstracción de datos: `ClockDisplay`

```java
/**
 * The ClockDisplay class implements a digital clock display for a
 * European-style 24 hour clock. The clock shows hours and minutes. The
 * range of the clock is 00:00 (midnight) to 23:59 (one minute before midnight).
 * The clock display receives "ticks" (via the timeTick method) every minute
 * and reacts by incrementing the display. This is done in the usual clock
 * fashion: the hour increments when the minutes roll over to zero.
 */
public class ClockDisplay
{
    /**
     * @post Creates a new clock set at 00:00.
     */
    public ClockDisplay()
    /**
     * @pre {@literal 0 <= 'hour' < 24 and 0 <= 'minute' < 60.}
     * @post Creates a new clock set at the time specified by the parameters.
     */
    public ClockDisplay(int hour, int minute)
    /**
     * @post Makes the clock display go one minute forward
     * (should get called once every minute).
     */
    public void timeTick()
    /**
     * @post Returns the current time of this display in the format HH:MM.
     */
    public String getTime()
    /**
     * @post Update the internal string that represents the display.
     */
    private void updateDisplay()
}
```

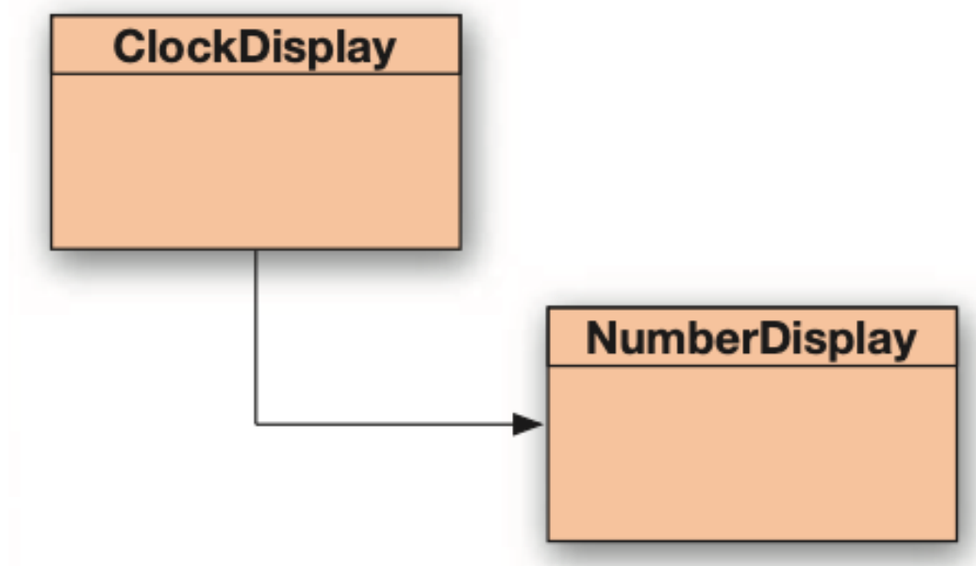
# Abstracción de datos: `ClockDisplay`

```
/**
 * The ClockDisplay class implements a digital clock display for a
 * European-style 24 hour clock. The clock shows hours and minutes. The
 * range of the clock is 00:00 (midnight) to 23:59 (one minute before midnight).
 * The clock display receives "ticks" (via the timeTick method) every minute
 * and reacts by incrementing the display. This is done in the usual clock
 * fashion: the hour increments when the minutes roll over to zero.
 */
public class ClockDisplay
{
    /**
     * @post Creates a new clock set at 00:00.
     */
    public ClockDisplay()
    /**
     * @pre {@literal 0 <= 'hour' < 24 and 0 <= 'minute' < 60.}
     * @post Creates a new clock set at the time specified by the parameters.
     */
    public ClockDisplay(int hour, int minute)
    /**
     * @post Makes the clock display go one minute forward
     * (should get called once every minute).
     */
    public void timeTick()
    /**
     * @post Returns the current time of this display in the format HH:MM.
     */
    public String getTime()
}
```

Aquí tenemos la especificación y las funcionalidades más importantes de `ClockDisplay`

# Diagrama de clases

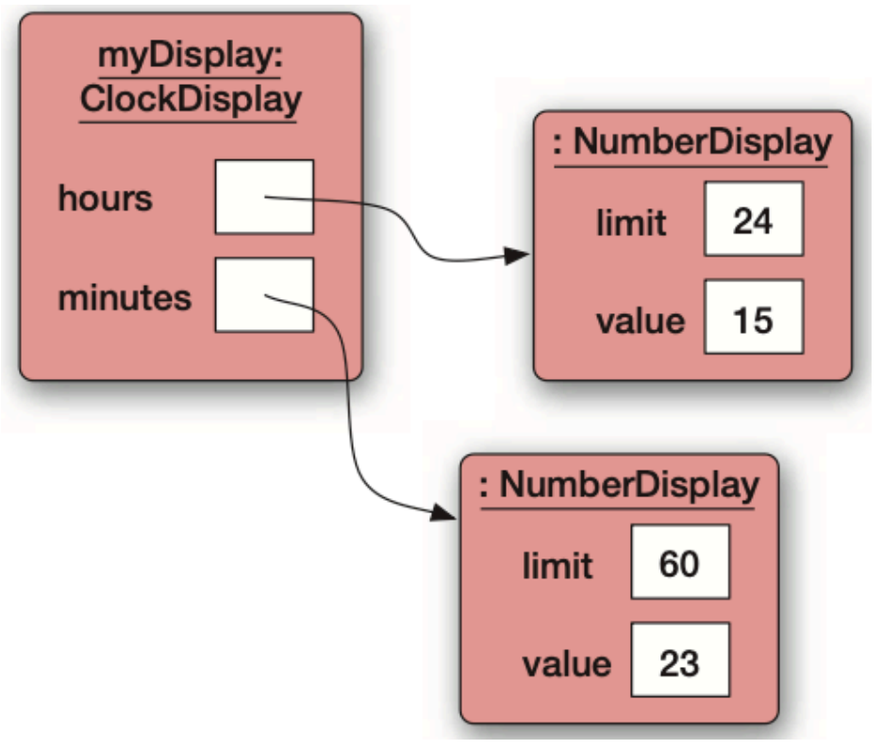
- Un diagrama de clases muestra las clases de la aplicación y la relación entre ellas
  - Muestra como está organizado el código de nuestro programa
  - Es decir, provee una vista estática del sistema
- En el diagrama, las clases se grafican con rectángulos
- Las flechas indican relaciones de uso
  - En el ejemplo, el código de la clase `ClockDisplay` usa funcionalidades de `NumberDisplay`




- Para entender un programa orientado a objetos es muy importante conocer el diagrama de clases, y las responsabilidades de las clases (qué función cumple cada clase, descrita brevemente en el comentario previo al código)

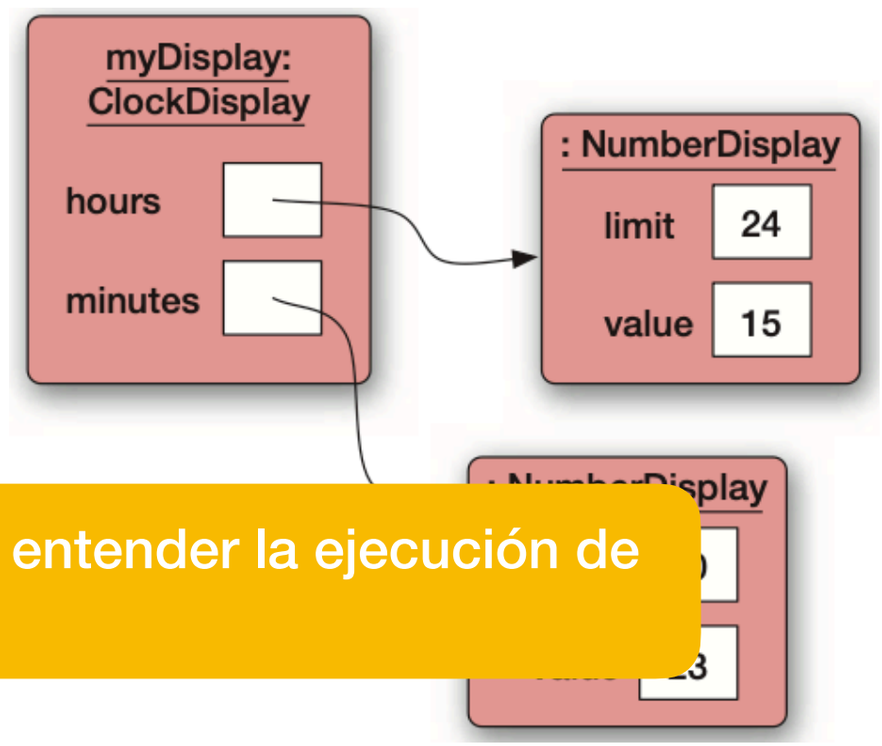
# Diagrama de objetos

- El diagrama de objetos muestra los objetos y las relaciones entre ellos en un momento dado de la ejecución del programa
  - Es decir, el diagrama de objetos muestra una vista dinámica del sistema
- Los diagramas de objetos son una herramienta muy útil para entender la ejecución de un programa OO
- Los rectángulos del diagrama representan objetos
  - Tienen espacio reservado para cada uno de sus atributos
- Una flecha en el diagrama indica que un atributo almacena una referencia a un objeto
- En el gráfico:
  - `hours` de `myDisplay` tiene una referencia a un objeto `NumberDisplay` con el valor `15`
  - `minutes` de `myDisplay` tiene una referencia a un objeto `NumberDisplay` con el valor `23`
  - Es decir, el reloj `myDisplay` marca las `15:23`



# Diagrama de objetos

- El diagrama de objetos muestra los objetos y las relaciones entre ellos en un momento dado de la ejecución del programa
  - Es decir, el diagrama de objetos muestra una vista dinámica del sistema
- Los diagramas de objetos son una herramienta muy útil para entender la ejecución de un programa OO
- Los rectángulos del diagrama representan objetos
  - Tienen espacio reservado para cada uno de sus atributos
- Una flecha en el diagrama indica que un atributo almacena una referencia a un objeto
- En el gráfico:
  - `hours` de `myDisplay` tiene una referencia a un objeto `NumberDisplay` con el valor `15`
  - `minutes` de `myDisplay` tiene una referencia a

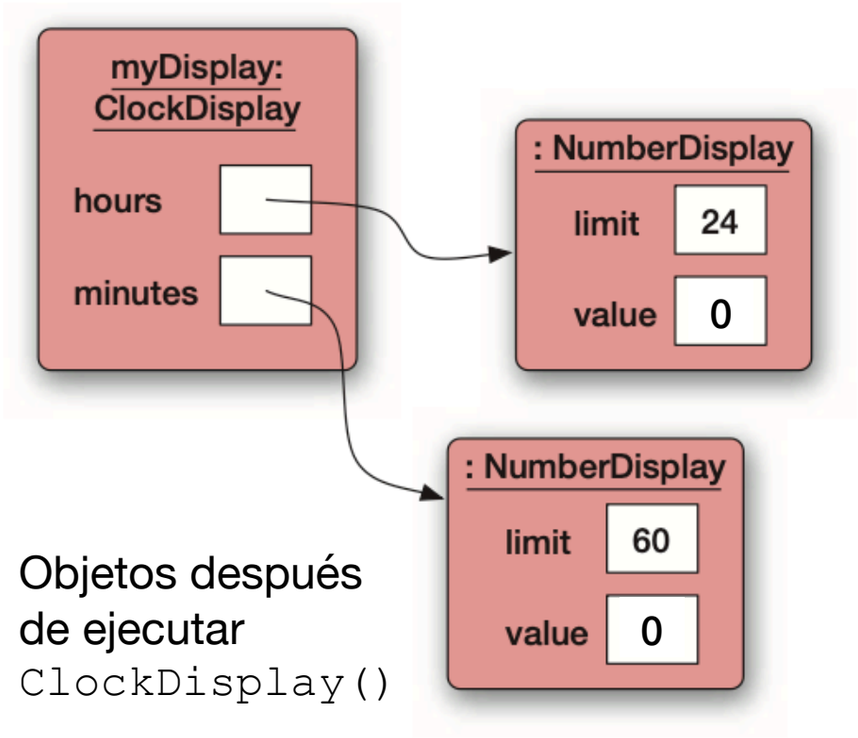



**Se recomienda dibujar los diagramas de objetos para entender la ejecución de sus programas**

# Objetos que crean objetos

```java
public class ClockDisplay
{
    private NumberDisplay hours;
    private NumberDisplay minutes;
    private String displayString;

    /**
     * @post Creates a new clock set at 00:00.
     */
    public ClockDisplay()
    {
        hours = new NumberDisplay(24);
        minutes = new NumberDisplay(60);
        updateDisplay();
    }
```




Objetos después de ejecutar `ClockDisplay()`

- Estamos listos para definir nuestro `ClockDisplay`, usando funcionalidades de `NumberDisplay`
- `ClockDisplay` tiene dos atributos de tipo `NumberDisplay`
  - Inicialmente, estos atributos son `null`
  - Para que estos atributos hagan referencia a objetos tenemos que crearlos primero
- El constructor de `ClockDisplay` crea dos objetos de tipo `NumberDisplay`, que luego la clase usará para definir su propia funcionalidad

# Invocación a métodos internos

- Para mantener el ejemplo simple, vamos a representar el valor del display con un atributo de tipo `String` (`displayString`)
- La última línea del constructor invoca a un método privado de la misma clase: `updateDisplay()`
  - Esto se llama una invocación a un método interno
- La invocación a un método interno no requiere que especifiquemos el objeto al cual se aplica
  - El método se ejecuta sobre el objeto actual
- Los métodos internos son por lo general privados (`private`)
  - Sólo pueden ser invocados desde la clase que los define
  - Definen funcionalidades útiles sólo para la clase

```
public class ClockDisplay
{
   private NumberDisplay hours;
   private NumberDisplay minutes;
   private String displayString;

   /**
    * @post Creates a new clock set at 00:00.
    */
   public ClockDisplay()
   {
       hours = new NumberDisplay(24);
       minutes = new NumberDisplay(60);
       updateDisplay();
   }

   /**
    * @post Update the internal string that represents
    * the display.
    */
   private void updateDisplay()
   {
       displayString = hours.getDisplayValue() + ":" +
                       minutes.getDisplayValue();
   }
```

# Invocación a métodos externos

- El operador `.` de Java puede usarse para invocar a métodos de otros objetos
- Por ejemplo, el código de `timeTick()` invoca a:
    - `minutes.increment()`
    - `minutes.getValue()`
    - `hours.increment()`
- Las llamadas a métodos de otros objetos se denominan invocaciones a métodos externos
- Notar que la responsabilidad de llevar el tiempo está dividida entre las clases `NumberDisplay` y `ClockDisplay`
    - Esto es un ejemplo de divide and conquer

```java
/**
 * @post Makes the clock display go one minute forward
 * (should get called once every minute).
 */
public void timeTick()
{
    minutes.increment();
    if(minutes.getValue() == 0) {
        // it just rolled over!
         hours.increment();
    }
    updateDisplay();
}
```

- `timeTick()` incrementa en 1 los minutos (`minutes.increment()`), y cada vez que vuelven a 0 incrementa e 1 la hora (`hours.increment()`)

# Múltiples constructores

- Notar que una clase puede tener más de un constructor
- En este ejemplo, tiene uno que inicializa el display por defecto (en 00:00), y otro que lo inicializa en un tiempo dado como parámetro
- Java soporta la sobrecarga de operadores: puede haber más de un constructor, o más de un método con el mismo nombre, siempre que tengan distintos  parámetros

```
public class ClockDisplay
{
    private NumberDisplay hours;
    private NumberDisplay minutes;
    private String displayString;

    /**
     * @post Creates a new clock set at 00:00.
     */
    public ClockDisplay()
    {
        hours = new NumberDisplay(24);
        minutes = new NumberDisplay(60);
        updateDisplay();
    }

    /**
     * @pre {@literal 0 <= 'hour' < 24 and 0 <= 'minu
     * @post Creates a new clock set at the time spec
     * parameters.
     */
    public ClockDisplay(int hour, int minute)
    {
        hours = new NumberDisplay(24);
        minutes = new NumberDisplay(60);
        setTime(hour, minute);
    }
```

# ClockDisplay: otros métodos

- Finalmente, podemos definir métodos de `ClockDisplay` para cambiar el tiempo del reloj, y para retornar el valor del display

```
/**
 * @pre {@literal 0 <= 'hour' < 24 and 0 <= 'minute' < 60.}
 * @post Set the time of the display to the specified hour and
 * minute.
 */
public void setTime(int hour, int minute)
{
    hours.setValue(hour);
    minutes.setValue(minute);
    updateDisplay();
}

/**
 * @post Returns the current time of this display in the format HH:MM.
 */
public String getTime()
{
    return displayString;
}
```

# Interfaz de una clase o módulo

- En proyectos del mundo real distintas personas suelen escribir el código de distintos módulos
- Para que esto sea posible, los desarrolladores tienen que ponerse de acuerdo en las funcionalidades que proveerá cada módulo
- De esta manera, un grupo de desarrolladores puede concentrarse en implementar un módulo, que los restantes desarrolladores pueden luego usar para implementar otros módulos
- La interfaz de una clase/módulo define las funcionalidades que provee la clase/módulo
  - La interfaz de la clase/módulo está conformada por sus métodos públicos
  - Por ejemplo, la interfaz de `ClockDisplay` es:

***Constructor Summary***

**Constructors**

| Constructor |
| --- |
| `ClockDisplay()` |
| `ClockDisplay(int hour, int minute)` |

***Method Summary***

**All Methods** | **Instance Methods** | **Concrete Metho**

| Modifier and Type | Method |
| --- | --- |
| `String` | `getTime()` |
| `void` | `setTime(int hour, int minute)` |
| `void` | `timeTick()` |

- Hablaremos mucho más sobre las interfaces más adelante en el curso...

# Resumen

- El objetivo de la modularización es dividir un problema complejo en partes más fáciles de resolver
- Definimos una abstracción de datos `NumberDisplay`, que luego nos facilita la implementación de `ClockDisplay`
- Al usar `NumberDisplay` empleamos la técnica de abstracción: sólo nos basamos en las especificaciones de los métodos e ignoramos la implementación
- Es mucho más fácil invocar a `increment` y asumir que el display se va a incrementar correctamente que hacer las cuentas matemáticas implementadas por `increment` (que involucran el reseteo)

```
public class NumberDisplay
{
    /**
     * @pre {@literal 'rollOverLimit' > 0}
     * @post Constructs a NumberDisplay that
     *     and resets at the given 'rollOver
     */
    public NumberDisplay(int rollOverLimit)
    /**
     * @post Return the display value, that
     *     as a two-digit String.
     */
    public String getDisplayValue()
    /**
     * @post Return the current value.
     */
    public int getValue()
    /**
     * @post Increment the display value by
     * limit is reached.
     */
    public void increment()
}
```

# Resumen

- Usando `NumberDisplay` la implementación de `ClockDisplay` es muy sencilla
- Como ejemplo, veamos el código de `timeTick`
- Notar que este código es muy fácil de leer, ya que no requiere pensar en operaciones aritméticas de bajo nivel, sino en operaciones abstractas fáciles de entender
- Si el código es fácil de entender, es mucho menos probable que contenga defectos, y es más fácil de modificar y extender

```java
public class ClockDisplay
{
    private NumberDisplay hours;
    private NumberDisplay minutes;
    private String displayString;

    /** código omitido **/

    /**
     * @post Makes the clock display go one mi
     * (should get called once every minute).
     */
    public void timeTick()
    {
        minutes.increment();
        if(minutes.getValue() == 0) {
            // it just rolled over!
            hours.increment();
        }
        updateDisplay();
    }

    /** código omitido **/
```

# Resumen

- Usando `NumberDisplay` la implementación de `ClockDisplay` es muy sencilla
- Como ejemplo, veamos el código de `timeTick`
- Notar que este código es muy fácil de leer, ya que no requiere pensar en operaciones aritméticas de bajo nivel, sino en operaciones abstractas fáciles de entender
- Si el código es fácil de entender, es mucho menos probable que

```java
public class ClockDisplay
{
    private NumberDisplay hours;
    private NumberDisplay minutes;
    private String displayString;

    /** código omitido **/

    /**
     * @post Makes the clock display go one mi
     * (should get called once every minute).
     */
    public void timeTick()
    {
        minutes.increment();
        if(minutes.getValue() == 0) {
            // it just rolled over!
            hours.increment();
        }
```

Ejercicio: Agregar los segundos a ClockDisplay, y testear la implementación

# Actividades

- Leer el capítulo 3 del libro "*Objects First with Java A Practical Introduction using BlueJ*". Sixth Edition. D. Barnes & M. Kölling. Pearson. 2016

- Leer el capítulo 5 del libro "*Program Development in Java - Abstraction, Specification, and Object-Oriented Design*". B. Liskov & J. Guttag. Addison-Wesley. 2001

# Bibliografía


- "*Objects First with Java A Practical Introduction using BlueJ*". Sixth Edition. D. Barnes & M. Kölling. Pearson. 2016

- "*Program Development in Java - Abstraction, Specification, and Object-Oriented Design*". B. Liskov & J. Guttag. Addison-Wesley. 2001